



UNIVERSIDAD  
**NACIONAL**  
DE COLOMBIA

## Laboratory 7 Scalability

Jeisson Andrés Vergara Vargas

Software Architecture

2026-I

---

# 1. Objective

---

Build and deploy a system that demonstrates **horizontal scaling** as a strategy to improve the performance and availability quality attributes, implementing:

- **Horizontal Scaling:** deploying multiple identical replicas of a stateless service using Docker Compose.
- **Load Balancing:** distributing incoming requests across replicas using Nginx, configuring and comparing three balancing algorithms: Round Robin, Least Connections, and IP Hash.
- **Instance Identification:** exposing the active replica's identity in each response, so the distribution effect is directly observable.

## 2. Context

---

### 2.1. Horizontal Scaling

**Horizontal scaling** (scale-out) is the architectural tactic of increasing capacity by adding more instances of a component, as opposed to *vertical scaling* (scale-up), which increases the resources of a single instance. In a horizontally scaled system, instances are typically **stateless**: each request can be handled by any replica without knowledge of prior interactions, because no session state is stored locally in the instance.

**Key implication:** statefulness is the main barrier to horizontal scaling. Services that store session data in memory cannot be freely replicated without a shared state store (e.g., Redis, a database). In this lab the service is intentionally stateless.

## 2.2. Load Balancing

A **load balancer** sits in front of a pool of replicas and routes each incoming request to one of them according to a *balancing algorithm*. In this lab Nginx acts as the load balancer. The three upstream algorithms explored are:

| Algorithm         | Nginx directive               | How it works                                                                                | Best suited for                                                            |
|-------------------|-------------------------------|---------------------------------------------------------------------------------------------|----------------------------------------------------------------------------|
| Round Robin       | <i>(default, none needed)</i> | Requests are forwarded to replicas in strict rotation: 1 → 2 → 3 → 1 → ...                  | Homogeneous replicas with similar request cost.                            |
| Least Connections | <code>least_conn</code>       | Each new request goes to the replica with the fewest active connections at that moment.     | Mixed workloads where some requests take longer than others.               |
| IP Hash           | <code>ip_hash</code>          | The client IP is hashed; the same client always reaches the same replica (sticky sessions). | Applications that require session affinity without a shared session store. |

## 2.3. Relationship to Quality Attributes

**Performance** improves because the total request throughput is distributed across replicas: where a single instance handles  $N$  requests/second,  $k$  replicas can handle up to  $k \times N$ . **Availability** improves because the failure of one replica does not take the entire service down — the load balancer continues routing traffic to the remaining healthy instances.

## 3. Prerequisites

- Unix-based OS (or WSL 2 on Windows).
- [Docker](#) and Docker Compose installed.
- `curl` available in the terminal (pre-installed on most systems).
- Optional: [Apache Bench \(ab\)](#) for load testing (Section 9.4).

## 4. Project Structure

Create a folder **scaling-patterns** with the following structure:

```
scaling-patterns/  
├── backend/  
│   ├── app.py  
│   └── Dockerfile  
├── nginx/  
│   ├── nginx.round-robin.conf  
│   └── nginx.least-conn.conf
```

```
└─ nginx.ip-hash.conf
└─ docker-compose.yml
```

## 5. Backend Service

The backend is a stateless Python/Flask application. Each replica exposes its own **hostname** and a simulated **processing time** in every response, making it easy to observe which instance handled each request and to reason about load distribution.

a. Into the **backend** folder, create an **app.py** file:

```
import os, socket, time, random
from flask import Flask, jsonify

app = Flask(__name__)

# Each container resolves its own hostname at startup.
# When Docker Compose creates multiple replicas, each gets
# a unique container name (backend_1, backend_2, backend_3).
INSTANCE_ID = socket.gethostname()

@app.route("/api/hello")
def hello():
    # Simulate variable processing time (50-300 ms)
    # to make least_conn behaviour observable.
    delay = random.uniform(0.05, 0.30)
    time.sleep(delay)
    return jsonify({
        "instance": INSTANCE_ID,
        "message": "Hello from the backend pool!",
        "processing_ms": round(delay * 1000, 1)
    })

@app.route("/api/status")
def status():
    return jsonify({
        "instance": INSTANCE_ID,
        "status": "healthy"
    })

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

b. Into the **backend** folder, create a **Dockerfile** file:

```
FROM python:3.11-slim

WORKDIR /app
```

```
RUN pip install flask

COPY app.py .

CMD ["python", "app.py"]
```

## 6. Load Balancer Configuration (Nginx)

---

Three separate Nginx configuration files are provided — one per algorithm. The only structural difference between them is the directive inside the `upstream` block. Read each carefully before switching between them in Section 9.

### 6.1. Round Robin (default)

No explicit directive is needed. Nginx distributes requests to each upstream server in strict rotation.

Into the **nginx** folder, create **nginx.round-robin.conf**:

```
events {}

http {

    upstream backend_pool {
        # Round Robin is the default: no extra directive needed.
        # Requests cycle: backend_1 → backend_2 → backend_3 → backend_1 →
        ...
        server backend_1:5000;
        server backend_2:5000;
        server backend_3:5000;
    }

    server {
        listen 80;
        server_name localhost;

        location /api/ {
            proxy_pass http://backend_pool;
            proxy_set_header Host $host;
            proxy_set_header X-Real-IP $remote_addr;
        }
    }
}
```

### 6.2. Least Connections

The `least_conn` directive routes each new request to the replica with the fewest active connections. This is more equitable when request processing times vary — a busy replica receives fewer new requests

while it is still handling previous ones.

Into the **nginx** folder, create **nginx.least-conn.conf**:

```
events {}

http {

    upstream backend_pool {
        least_conn; # Route to the replica with fewest active connections.
        server backend_1:5000;
        server backend_2:5000;
        server backend_3:5000;
    }

    server {
        listen 80;
        server_name localhost;

        location /api/ {
            proxy_pass http://backend_pool;
            proxy_set_header Host $host;
            proxy_set_header X-Real-IP $remote_addr;
        }
    }
}
```

## 6.3. IP Hash

The **ip\_hash** directive computes a hash of the client's IP address and always maps it to the same upstream server. This provides *sticky sessions*: the same client will always reach the same replica, as long as that replica is available.

Into the **nginx** folder, create **nginx.ip-hash.conf**:

```
events {}

http {

    upstream backend_pool {
        ip_hash; # A given client IP always reaches the same replica.
        server backend_1:5000;
        server backend_2:5000;
        server backend_3:5000;
    }

    server {
        listen 80;
        server_name localhost;
    }
}
```

```

        location /api/ {
            proxy_pass http://backend_pool;
            proxy_set_header Host $host;
            proxy_set_header X-Real-IP $remote_addr;
        }
    }
}

```

## 7. Deployment with Docker Compose

---

a. Into the **scaling-patterns** folder, create a **docker-compose.yml** file:

```

services:

  backend_1:
    build: ./backend
    container_name: backend_1
    networks:
      - backend-net
    # No ports exposed: unreachable from the host directly

  backend_2:
    build: ./backend
    container_name: backend_2
    networks:
      - backend-net

  backend_3:
    build: ./backend
    container_name: backend_3
    networks:
      - backend-net

  load-balancer:
    image: nginx:alpine
    container_name: load-balancer
    ports:
      - "80:80"
    volumes:
      # Mount one configuration at a time. Start with round-robin:
      - ./nginx/nginx.round-robin.conf:/etc/nginx/nginx.conf:ro
    networks:
      - backend-net
    depends_on:
      - backend_1
      - backend_2
      - backend_3

networks:

```

```
backend-net:
  driver: bridge
```

Key observations:

- The three **backend** services share a single **backend-net** network. None of them has a **ports** mapping, meaning they are unreachable from the host or the outside world — only the load balancer can reach them.
- **load-balancer** is the sole entry point, exposing port **80** to the host.
- Three backend replicas are declared as **separate named services** (**backend\_1**, **backend\_2**, **backend\_3**). This keeps container names predictable, which is required so that Nginx upstream hostnames (**backend\_1:5000**, etc.) resolve correctly inside the Docker network.

**b.** Build and start the system:

```
docker compose up --build -d
```

Verify all four containers are running:

```
docker compose ps
```

## 8. Scaling Operations

---

### 8.1. Scaling Down: Stopping a Replica

Stop one replica to simulate an instance failure and observe that the load balancer continues serving traffic through the remaining two:

```
# Stop one replica
docker compose stop backend_3

# Only backend_1 and backend_2 should now appear in responses
for i in $(seq 1 6); do
  curl -s http://localhost/api/hello | python3 -c \
    "import sys, json; d = json.load(sys.stdin); print(d['instance'])"
done
```

### 8.2. Scaling Back Up: Restoring a Replica

```
# Restart the stopped replica
docker compose start backend_3
```

```
# Confirm it re-enters the rotation
for i in $(seq 1 9); do
    curl -s http://localhost/api/hello | python3 -c \
        "import sys, json; d = json.load(sys.stdin); print(d['instance'])"
done
```

## 8.3. Switching the Balancing Algorithm

To switch algorithms, edit the `volumes` mount in `docker-compose.yml` and recreate only the load balancer container (the backend containers keep running without interruption):

```
# 1. Edit docker-compose.yml. Change the volume line to the desired config,
e.g.:
#     - ./nginx/nginx.least-conn.conf:/etc/nginx/nginx.conf:ro

# 2. Recreate only the load-balancer (--no-deps leaves backends untouched)
docker compose up -d --no-deps load-balancer

# 3. Confirm Nginx loaded the new configuration without errors
docker logs load-balancer
```

## 9. Testing

---

### 9.1. Basic Round-Trip

```
curl -s http://localhost/api/hello
```

Expected response (instance name will vary):

```
{
  "instance": "backend_2",
  "message": "Hello from the backend pool!",
  "processing_ms": 173.4
}
```

### 9.2. Observing Round-Robin Distribution

Send nine consecutive requests. Each backend should appear approximately three times:

```
for i in $(seq 1 9); do
    curl -s http://localhost/api/hello | python3 -c \
```



```
"import sys, json; d = json.load(sys.stdin); print(d['instance'])"
done
```

Expected output pattern:

```
backend_1
backend_2
backend_3
backend_1
backend_2
backend_3
backend_1
backend_2
backend_3
```

## 9.3. Observing IP Hash Stickiness

After switching to `ip_hash` (Section 8.3), all requests from the same machine should always return the same `instance` value:

```
for i in $(seq 1 6); do
  curl -s http://localhost/api/hello | python3 -c \
    "import sys, json; d = json.load(sys.stdin); print(d['instance'])"
done
# All lines should show the same instance name.
```

## 9.4. Load Test with Apache Bench (optional)

Send 300 requests with a concurrency of 10 and measure throughput:

```
ab -n 300 -c 10 http://localhost/api/hello
```

Key metrics to note: *Requests per second*, *Time per request (mean)*, and *Failed requests* (should be 0). Repeat after scaling down to two replicas (`docker compose stop backend_3`) and compare the results.

## 9.5. Verifying Backend Isolation

Confirm that backend containers are not directly reachable from the host:

```
curl http://localhost:5000/api/status
# Expected: Connection refused – port 5000 is not exposed.
```

## 9.6. Inspecting the Upstream Pool at Runtime

```
# Verify Nginx sees all three upstreams in the active configuration
docker exec load-balancer nginx -T | grep server

# Tail the Nginx access log to watch distribution in real time
docker logs -f load-balancer
```